FACULTAD DE
INGENIERÍA
UNIVERSIDAD DE SANTIAGO DE CHILE

Laboratorio 2

Análisis de estrategias algorítmicas

Diseño de Algoritmos (13346-0-B-2)

Integrante: Joaquin Alvarez

04 de marzo del 2026

Metodología

El entorno virtual utilizado fue WSL (Windows Subsystem for Linux), el código sigue el estándar C11 por si es compilado con MSVC. Se ejecutaron benchmarks para cada uno de los algoritmos siguiendo el procedimiento de realizar 10 ejecuciones utilizando los mismos parámetros y luego determinar el tiempo promedio.

Los valores de n y T se ajustaron para cada algoritmo y se buscó observar las limitaciones de cada uno de estos y se realizaron diferentes pruebas al encontrarse con una limitación nueva.

Para los valores de $t(i)$ y $p(i)$ se utilizo la funcion auxiliar $random_array(int\ n, int\ min, int\ \max)$ para generar un arreglo de enteros aleatorios, cada número entero se forma usando otra función auxiliar llamada $random_int(int\ min, int\ max)$ que genera un numero entero que llamaremos a, que está dentro de un rango $min \leq a \leq max$. Esta forma de crear los arreglos $t(i)$ y $p(i)$ será utilizada a menos que haya una dependencia en la distribución de estos datos.

Se considera que el problema planteado es una variación del problema de la mochila 0/1 y se mencionara igualmente su versión de la mochila fraccionaria.

Justificación del criterio goloso elegido y una descripción de los otros dos enfoques.

El criterio goloso escogido utiliza el ratio puntaje/tiempo para determinar cuál de todos los temas es el que utilizaría mejor el tiempo bajo este criterio. Luego de escoger el tema se decide si existe otro tema que tenga el mismo puntaje/tiempo y entre estos se elige el que tenga el mayor puntaje para cubrir el máximo tiempo posible con la mejor decisión actual.

Para el enfoque backtracking partimos de la base de un algoritmo de combinatoria básico. Tenemos 2 opciones, estudiamos el tema o no, esta es nuestra recursión. Primero tomar la decisión de si elegir el tema, ver si la solución era valida al final del árbol de recursión y luego hacer backtracking y no escoger el tema. El caso base será cuando nos quedemos sin temas que añadir $(i == n)$, estas serán las hojas del árbol de recursión. Verificamos si cumple las condiciones, actualizamos la mejor solución y retrocedemos para seguir buscando hasta quedarnos sin permutaciones sin ver.

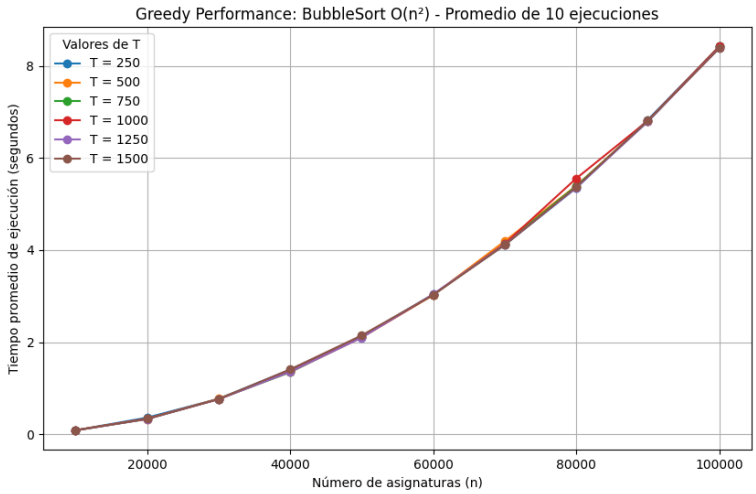
El enfoque programación dinámica utiliza el mismo caso base que backtracking de quedarse sin temas $(i == 0)$ pero añade otro caso base más que es quedarse sin

tiempo $(j == 0)$, todo esto en el ambiente del subproblema creado, y toma la mejor decisión entre estudiar el tema, básicamente sumándole su puntaje a la mejor opción donde el tiempo sea $j - t(i)$ y el tema actual no es considerado, donde j es el tiempo actual que estamos tomando para formar la solución $\left(DP\left(i-1, j-t(i)\right)\right)$, y saltarse ese tema, donde se toma la mejor opción donde no existía ese tema $(i-1)$ dentro del mismo tiempo j $\left(DP(i-1, j)\right)$.

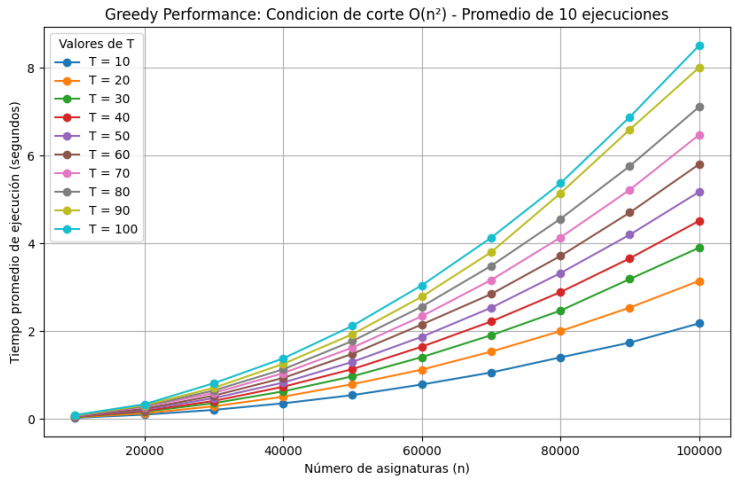
Análisis de complejidad temporal para todos los enfoques.

Enfoque goloso:

La versión inicial del algoritmo goloso tenía una complejidad temporal $O(n^2)$ por utilizar una variación del algoritmo BubbleSort para obtener el tema con el mayor puntaje/tiempo posible y esto era lo que tomaba más tiempo de realizar en valores de n grandes.



Luego de esto implemento un break con la condición de que mientras $time_accumulated < T$ se sigan introduciendo temas a la solución. Esto creo una relación entre el valor de T y el tiempo que nos tomaría estudiar todos los temas, es decir, la zona donde esta condición tiene un impacto se encuentra en $0 \leq T \leq \sum t(i)$, este siendo proporcional a que tan cerca este T de $\sum t(i)$



Esto fue efectivo en reducir la cantidad de ciclos al pasar por los arreglos ordenados según el criterio, y como ordenaba tema por tema a la vez que se construye la solución, también mejoro el tiempo del algoritmo proporcionalmente al tamaño de T, si todos los temas caben en el tiempo total, entonces se realizaran siempre todos los ciclos.

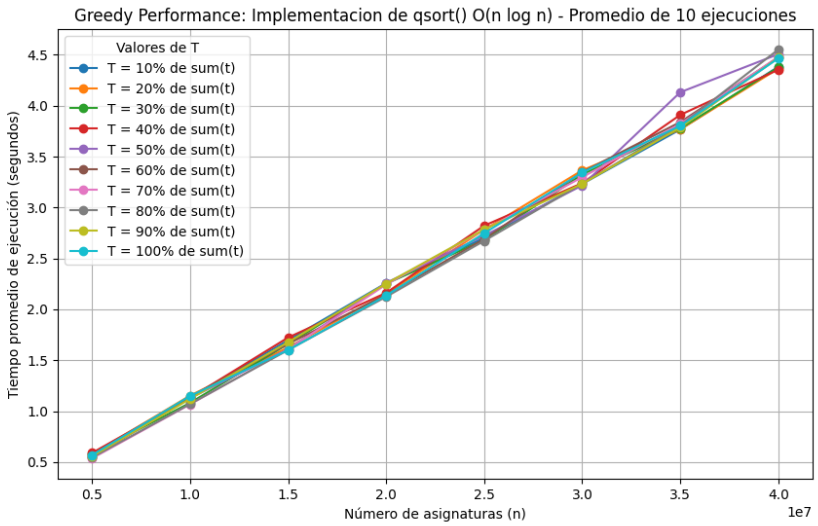
Ahora apuntando a otro claro problema, BubbleSort es nuestro cuello de botella actual, hay algoritmos de ordenamiento más rápidos con complejidad temporal $O(n\log(n))$.

La solución a esto fue la función qsort la cual viene en la C standard library (stdlib.h) por no necesitar de un arreglo temporal de tamaño n teniendo una complejidad espacial en el caso promedio $O(log(n))$ por la pila de recursión y junto con esto vino la idea de utilizar una estructura de datos que guardara el puntaje del tema, el tiempo de estudio, el índex del tema y el ratio puntaje/tiempo, ahora nos referiremos a esta estructura como Tema.

```
typedef struct Tema {
    int tiempo;
    int puntaje;
    int index;
    float ratio;
} Tema;
```

MergeSort tenía un problema al medirlo, necesitaba un memoria auxiliar y como estamos ordenando Tema's y cada una de estas pesa 16 bytes al ingresar $n's$ lo suficientemente grandes $n \geq 33554432$ el solo ejecutar MergeSort utilizara $+1\ GiB$ de memoria RAM.

Un arreglo de Tema's ordenado por ratio nos daría exactamente lo que se necesita para cumplir con el criterio goloso establecido y qsort tiene una complejidad temporal en el caso promedio $O(n\ log(n))$ pero viene con la contra de que tiene una complejidad $O(n^2)$ para su peor caso, aunque esto viene de compararlo con QuickSort y C standard library no menciona nada acerca de cómo funciona su implementación, solo que en algún momento fue Quicksort. La información recopilada al respecto menciona que qsort es un hibrido entre QuickSort, HeapSort y Insertion Sort para garantizar una complejidad temporal $O(n\ log(n))$ en el peor caso y esta es la tesis con la que seguiremos en adelante, qsort tiene una complejidad temporal $O(n\ log(n))$ y espacial $O(log(n))$.



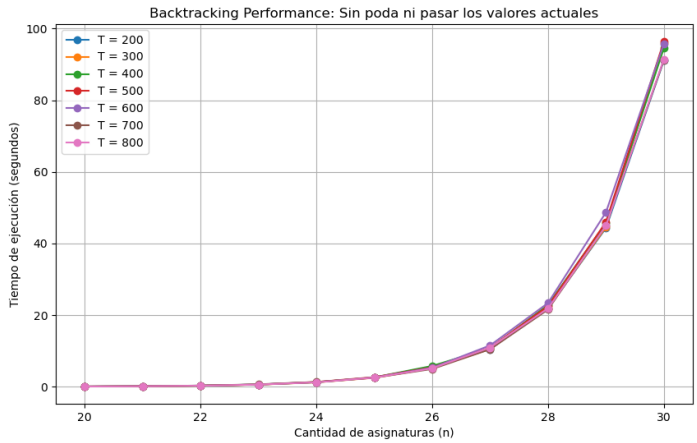
Como vemos en el gráfico, la condición que se impuso anteriormente ya no tiene tanto impacto debido a que no es posible de implementar en la parte más costosa del algoritmo (qsort) y cuando es implementada en el ciclo que crea la solución $O(n)$, el impacto de este es mínimo comparado con el ordenamiento, solo está asegurado que el arreglo de Tema's esta ordenado al final, por ende, no podemos crear una condición de este tipo que tenga impacto en valores de n grandes.

Por ende, la complejidad temporal del algoritmo goloso es de $O(n\ log(n))$ y una complejidad espacial $O(n)$ por el arreglo de Tema's.

Enfoque backtracking:

La versión inicial de este algoritmo fue uno de combinatoria simple donde se decidía si estudiar el tema o no y se pasaba al siguiente tema hasta quedarse sin temas ahí se verificaba su validez y se comparaba con la mejor solución encontrada previamente si

existía. Como hay 2 opciones por elemento y esto crea un árbol de recursión con altura $log_2(n)$ y con 2^n nodos, se recorren todos los nodos por ende el algoritmo tiene una complejidad temporal $O(2^n)$ y una complejidad espacial $O(n)$ para guardar la solución.



Vemos como la complejidad temporal se demuestra en las pruebas y luego veremos el impacto de la poda.

Enfoque programación dinámica:

Para el enfoque utilizando programación dinámica básicamente se recorre una matriz de tamaño $(n+1) \times (T+1)$ donde cada casilla corresponde a la solución de un subproblemas. Los subproblemas son resueltos con los casos base, los cuales son las soluciones de los subproblemas triviales. La solución se construye recorriendo la tabla $O(n \cdot T)$ y realizando comparaciones y asignaciones $O(1)$, así que el algoritmo tiene una complejidad temporal $O(n \cdot T)$ y una complejidad espacial $O(n \cdot T)$

Discusión sobre la correctitud de todos los enfoques.

Enfoque programación dinámica y backtracking:

Respecto a la correctitud de los enfoques tanto utilizando programación dinámica como backtracking se puede decir que ambos darán una de las soluciones optimas que maximiza $\sum p_i$ debido a su funcionamiento nato.

El enfoque de programación dinámica construye la solución a partir de las soluciones optimas de los subproblemas que constituyen el problema en sí, a través de las opciones, los casos base nos darán soluciones optimas que ya conocemos y con esto construimos las soluciones que no, estos son los subproblemas, asegurando la solución óptima.

El enfoque de backtracking es más simple, se buscan todas las posibles combinaciones de temas y al conseguir cada una se decide si es la óptima o no. Luego de revisar todas las combinaciones posibles (Que cumplan las condiciones, es decir, que "sobrevivan" la poda) solo queda la solución óptima, sabemos que maximiza el puntaje porque ya revisamos TODAS las posibles soluciones, asegurando su correctitud.

Enfoque goloso:

Aquí es donde está la diferencia entre este algoritmo y los demás, a pesar de ser el más rápido por un gran margen, este es incorrecto desde un punto de vista de obtener la solución óptima, pero genera soluciones validas que se acercan a la solución óptima y en casos si son la mejor solución.

El problema reside en que el algoritmo toma decisiones locales, parecido al enfoque backtracking, pero sin la posibilidad de dar pasos atrás para revisar si existe una solución mejor, obtiene la mejor solución a la primera con el criterio dado y esto no siempre funciona ya que solo revisa la decisión actual, no "piensa en el futuro" y esto provoca fallos, pero si el criterio fue el correcto, debería entregarnos una solución cercana al optimo.

Ej.

$$\boldsymbol{n = 3, T = 50}$$

$$\boldsymbol{p = [60, 100, 120]}$$

$$\boldsymbol{t = [10, 20, 30]}$$

Esto quiere decir que:

Tema 1: Puntaje 60 y toma 10 de tiempo (Ratio 6.0)

Tema 2: Puntaje 100 y toma 20 de tiempo (Ratio 5.0)

Tema 3: Puntaje 120 y toma 30 de tiempo (Ratio 4.0)

Según el criterio goloso escogido deberíamos estudiar el tema con el mayor ratio:

Escoge el tema 1: El T restante es 40, puntaje 60

Escoge el tema 2: El T restante es 20, puntaje 160

No hay espacio para el tema 3, T restante sigue siendo 20

El máximo puntaje fue de 160 pero esto valor claramente no es el óptimo ya que tomando el tema 2 y 3 podríamos haber obtenido 220 y ocupar todo el tiempo disponible.

El otro problema del algoritmo goloso puro puede alejarse demasiado del optimo si el primer tema que no se toma es muy valioso, esto nos dice que la solución golosa puede no estar ni cerca de la solución óptima.

Ej.

$$n = 2, T = 100$$

$$p = [1, 99]$$

$$t = [1, 100]$$

Sacando los ratios:

$$r = [2.0, 0.99]$$

Según nuestro criterio goloso, se escoge el mayor ratio:

Se escoge el tema 1: El T restante es 99, puntaje 1

No hay espacio para el tema 2, T restante sigue siendo 99

Sabemos que el óptimo en este caso debe ser 99, pero nuestro algoritmo falla en aproximadamente un 98.98% con respecto al optimo.

Se puede optimizar el resultado con cotas, más específicamente acotando con la solución de la mochila fraccionaria, nos asegurará que la solución golosa sea como mínimo un porcentaje del optimo poniendo una condición para que se considere igualmente el tema que más puntaje nos dará. Esto se puede demostrar matemáticamente (Se considera el problema planteado como el problema de la mochila 0/1):

Teniendo un arreglo ordenado por ratio r:

$$r_1 \leq r_2 \leq r_3 \cdots r_n$$

Sea k el primer elemento que no cabe tras suma los elementos anteriores. La solución de la mochila fraccionaria (V_{frac}) se calcula llenando la mochila con los primeros $k - 1$ elementos con una fracción del elemento k para completar la capacidad W.

$$V_{frac} = \sum_{i=1}^{k-1} v_i + \frac{v_k}{Capacidad\ restante}$$

Es una propiedad fundamental que la solución del problema de la mochila 0/1 (V_{opt}) jamás podrá superar a la fraccionaria (V_{frac}):

$$V_{opt} \leq V_{frac}$$

Y que la solución golosa del problema de la mochila (V_{greedy}) jamás superara el óptimo:

$$V_{greedy} \leq V_{opt}$$

Sabemos que V_{frac} es la suma de los elementos que si cupieron (V_{greedy}) más una parte del elemento k. Por lo tanto:

$$V_{greddy} = \sum_{i=1}^{k-1} v_i \longrightarrow V_{frac} < V_{greedy} + v_k$$

$$V_{opt} < V_{greedy} + v_k$$

Dividiendo todo en 2:

$$\frac{V_{opt}}{2} < \frac{V_{greedy} + v_k}{2}$$

Entonces podemos concluir que el promedio entre V_{greedy} y v_k es por lo menos, mayor al 50% de la solución óptima y con esto podemos llegar a que el máximo entre estos dos cumple la condición igualmente:

$$\frac{V_{opt}}{2} < \max(V_{greedy}, v_k) \leq V_{opt}$$

Pero existe una desigualdad que no hemos tomado en cuenta, como este acotamiento es débil ya que solo mira el elemento k, pudiendo existir un elemento que cumpla las mismas condiciones después de k. Aquí entra v_{max} que es el valor del elemento con el mayor valor que si cabe en la mochila por si solo y este impone la desigualdad:

$$v_k \leq v_{max}$$

Finalmente llegamos a una cota más robusta:

$$\max(\mathrm{V_{greedy}}, v_k) \leq \max\,(V_{greedy}, v_{max})$$

En conclusión:

$$\frac{V_{opt}}{2} < \max(V_{greedy}, v_{max}) \leq V_{opt}$$

Esta modificación en el algoritmo goloso nos asegura que la solución encontrada es como mínimo mayor al 50% del optimo del problema.

Lo que usamos fue una enumeración parcial de tamaño k, también llamado PTAS (Polinomial Time Approximation Scheme), más específicamente un PTAS con $k = 1$. Lo que hicimos fue que antes de aplicar el criterio goloso, probamos todas las combinaciones posibles de k elementos que quepan en la mochila, de ahí viene v_{max}.

Podemos incrementar k para incrementar el porcentaje asegurado respecto al optimo que nos podrá dar el algoritmo en el peor caso, siempre con un costo en tiempo. El tiempo que nos toma generar estas combinaciones es de $O(n^k)$ con k siendo una constante.

El problema es que siempre tendremos una aproximación asegurada mas nunca llegaremos al optimo asegurado, esto se puede notar más claramente en la fórmula de la $Garantía$ en el esquema de aproximación de Sartaj Sahni.

$$Garantía = (1 - \frac{1}{k-1}) \times Solucion_{real}$$

En practica esto se va de control bastante rápido, si queremos garantizar el 90% del optimo necesitamos obtener primero el mejor valor de todas las combinaciones de 11 elementos, esto haría que la complejidad temporal del algoritmo sea $O(n^{11})$ lo cual es bastante lento a pesar de ser polinomial. Aun así, no necesitamos mantener esas combinaciones en memoria por ende la complejidad espacial se mantiene.

El algoritmo goloso presentado utilizara un PTAS con $k = 1$ para no afectar la complejidad temporal original mejorando su optimalidad.

Explicación formal del modelo de programación dinámica.

Este modelo de programación dinámica se basa en construir una tabla (matriz) donde cada celda representa la solución óptima de un subproblema, a esta tabla la llamaremos DP durante la explicación.

Definiremos $DP(i, j)$ como el puntaje máximo que se puede obtener tomando en cuenta los temas desde 1 hasta i con un tiempo disponible j.

$$i \epsilon [1, n],\ i \in \mathbb{N} \rightarrow \text{ Índex del tema actual}$$

$$j \epsilon [1, T],\ j \in \mathbb{N} \rightarrow \text{Tiempo disponible actual}$$

Ecuación de Bellman:

El puntaje en cada celda $DP(i,j)$ se calcula basándose en las soluciones obtenidas del tema anterior $(i-1)$. Para el tema i, hay 2 posibilidades que ya mencionamos antes.

- **No estudiar el tema i:** El puntaje es el mismo que el obtenido tomando en cuenta los primeros $i-1$ temas anteriores.
- **Estudiar el tema i:** Solo si el tiempo que toma estudiar el tema (t_i) es menor o igual al tiempo disponible actual j. Si incluimos el tema, sumamos el puntaje p_i al puntaje optimo obtenido con el tiempo disponible restante $(j - t_i)$.

Es decir:

$$DP(i,j) = \begin{cases} DP(i-1,j) & if\ t_i > j \\ \max\left(DP(i-1,j), p_i + DP(i-1, j-t_i)\right) & if\ t_i \leq j \end{cases}$$

Aquí faltan los casos base, subproblemas de los cuales ya sabemos la solución y son triviales, estos son la fundación necesaria para inicial la recursión:

- **Si no hay temas $(i = 0)$:** El puntaje máximo posible es 0, sin importar el tiempo disponible.
- **Si no hay tiempo $(j = 0)$:** El puntaje máximo posible es 0, no se puede estudiar ningún tema.

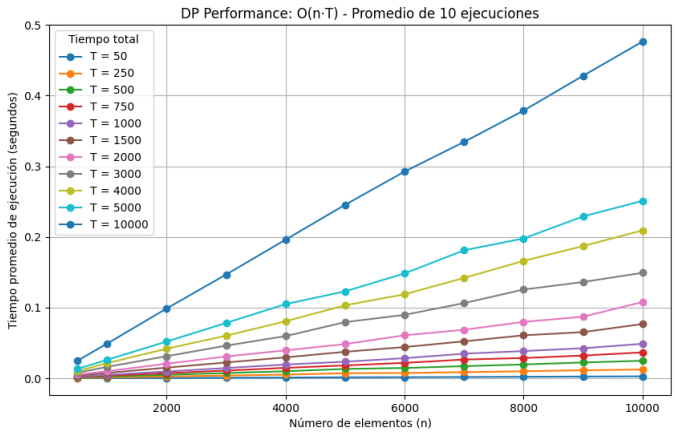
Juntando todas estas condiciones tenemos la ecuación de Bellman para este problema.

Formalmente:

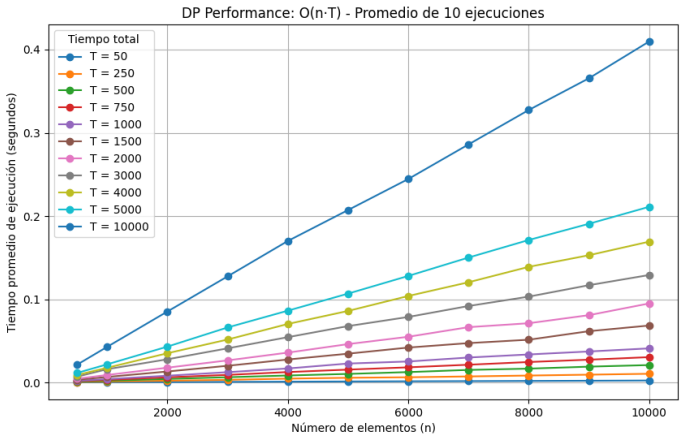
$$DP(i,j) = \begin{cases} 0 & if\ i = 0 \\ 0 & if\ j = 0 \\ DP(i-1,j) & if\ (i,j) > (0,0)\ \wedge\ t_i > j \\ \max\left(DP(i-1,j), p_i + DP(i-1, j-t_i)\right) & else \end{cases}$$

Pequeña mejora:

El algoritmo DP no es muy flexible respecto a cómo podemos optimizarlo, siempre requerirá recorrer la tabla.



En la imagen se puede notar la complejidad pseudo-polinomial que mencionamos antes.



Aquí mejoramos el tiempo general de ejecución en un 13% mediante inicializar punteros antes de realizar las comparaciones, esto para evitar que el procesador deba calcular primero donde existe $DP[i]$ o $DP[i-1]$ y aumentar el Hit Rate de la cache.

Limitaciones prácticas de cada solución.

Las limitaciones prácticas que se encontraron durante el testeo de cada uno de los enfoques son:

Algoritmo goloso:

La única limitación encontrada durante el testeo de este algoritmo fue que no otorga la solución óptima a cambio de ser el algoritmo más rápido, las limitaciones mencionadas anteriormente fueron corregidas mediante el uso de la función qsort() y la estructura de datos Tema.

Algoritmo backtracking:

Este algoritmo cuenta con la única limitación de no ser consistente, es imposible evitar su peor caso $O(2^n)$ pero hay maneras de reducir la búsqueda, podemos podar ramas o acotarla, siempre tomando en cuenta el verdadero tamaño del árbol de decisión con $2^{n+1} - 1$ nodos.

Algoritmo programación dinámica:

La principal limitación de este algoritmo es su dependencia de T, aunque el más consistente entre este mismo y el algoritmo backtracking, el peor caso sigue siendo $O(n \cdot T)$, hay que tener en cuenta que su complejidad sigue siendo pseudo-polinomial. Las consecuencias de esto se notan en la memoria al tener que guardar tal cantidad de enteros cuando T es demasiado grande o cuando $n \ll T$ el impacto sobre el tiempo de ejecución que tiene la cantidad de tiempo disponible que hay será mucho mayor a la cantidad de temas.

Aquí igual existe un esquema de aproximación tipo PTAS que hace que este algoritmo pueda llegar a ser más rápido y consuma menos RAM a cambio de la optimalidad del resultado, este es el FPTAS (Fully Polinomial Time Approximation Scheme), este traspone la matriz y hace que pasemos de tener un algoritmo con complejidad $O(n \times T)$ a $O(n^2 \times p_{max})$ y el problema de T pasa a ser de p_{maz}. Esto se mezcla con Profit Scaling donde lo que se hace es básicamente comprimir la matriz borrando información en el proceso (Truncando decimales) con un factor de compresión k que escala con ϵ que es el error máximo tolerable que escogemos.

$$k = \epsilon \times \frac{p_{max}}{N}$$

Dividiendo todos los puntajes por k:

$$p_i{}' = \left\lfloor \frac{p_i}{k} \right\rfloor$$

Con esto el nuevo puntaje máximo es:

$$p'_{max} = \frac{n}{\epsilon} \rightarrow Complejidad\ O(n^2 \times p_{max})\ pasa\ a\ ser\ O(n^3 \times \epsilon)$$

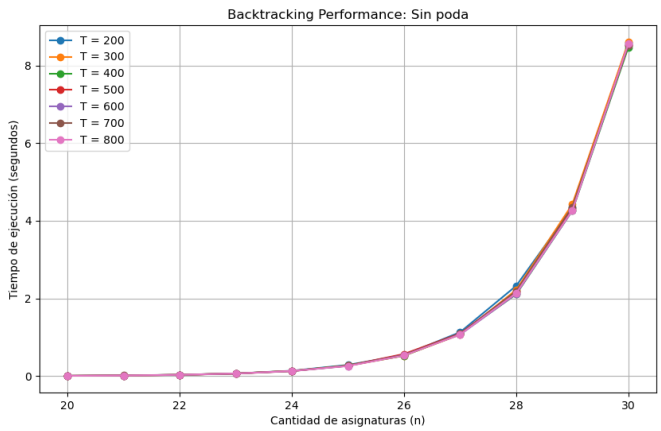
Siendo ϵ una constante, tenemos una complejidad completamente polinomial. Es mas lento que el goloso usando PTAS con precisiones bajas pero funciona mejor cuando quieres alta precisión sin aumentar el grado del polinomio.

No usare esta implementación de DP para este caso ya que asumo optimalidad con Programación Dinámica y FPTAS también intercambia optimalidad por un tiempo polinomial.

Implementación y mejora en la poda

Modificar pasar los valores necesarios a través de los argumentos:

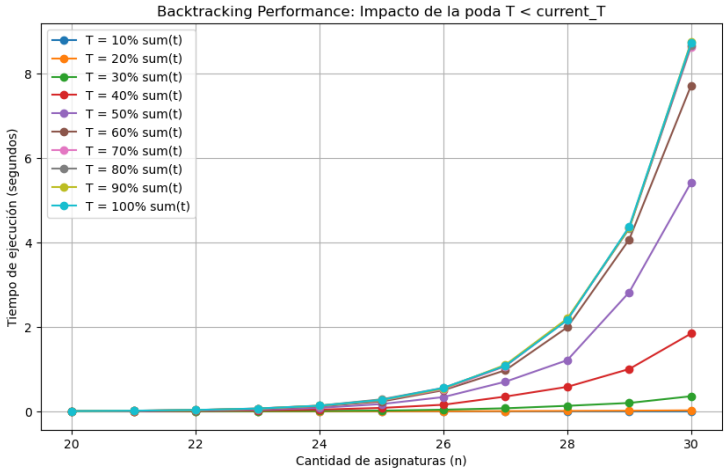
Primero está el hecho pasar tiempo usado en la rama actual y el puntaje obtenido actual ya que calcular este cuando ya tenemos la permutación completa hace que el algoritmo tenga una complejidad $O(2^n \cdot n)$ ya que se pasa por la permutación nuevamente cada vez que se completa teniendo un costo de $O(n)$ y el costo de pasarlo como argumento es de $O(1)$.



Esto mejoro considerablemente el tiempo de ejecución, pasamos de tomar casi 100[s] en ejecutar n = 30 a hacerlo en aproximadamente 8[s].

Poda (Pruning), evitar seguir buscando por vías invalidas.

Ahora la idea sería cortar las ramas que superen T por redundancia, esto quiere decir que para que las pruebas sean sensibles y nos den información para mejorar el algoritmo debemos situar T como un porcentaje de $\sum t(i)$, por ejemplo: Si T = $\sum t(i)$, entonces el algoritmo pasara por todas las ramas posibles, será como si nuestra poda no este, entonces nos mantendremos dentro de ese margen, $0 \leq T \leq \sum t(i)$.



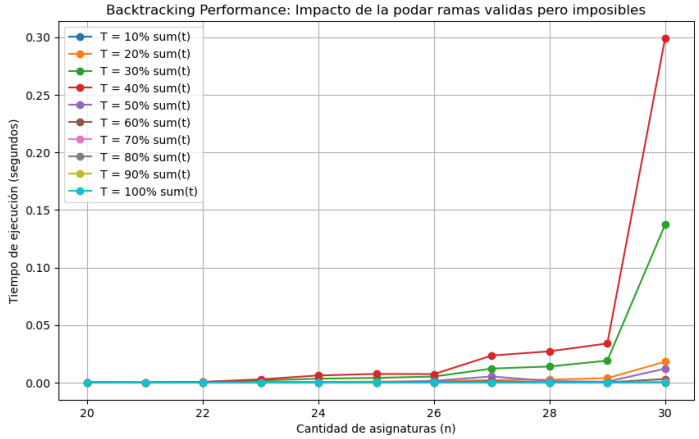
Como se puede notar, si $T \geq \sum t(i)$ entonces se evita completamente la poda y el rendimiento es igual que antes. Aun así, el margen de mejora se nota sobre todo cuando T es mucho menor a $\sum t(i)$.

Cotas (Bounds), reducir el área de búsqueda.

Como siguiente paso queremos eliminar aquellas ramas viables que no tienen futuro utilizando una cota para directamente jamás pasar por estas, esta será un arreglo de $n+1$ enteros que tendrá el máximo puntaje que se podría obtener seleccionando temas desde i hasta n.

Formalmente:

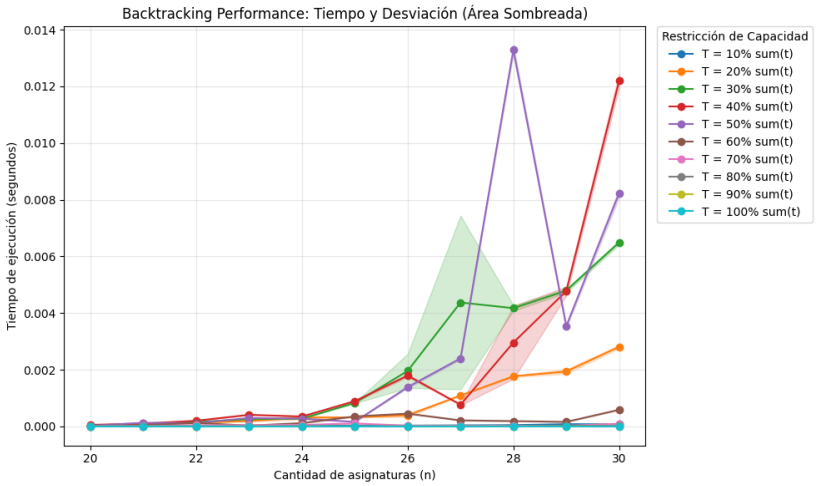
$$remainingSum(i) = \sum_{j=i}^{n-1} p_j$$



Un problema con esta implementación es que, al ver de cerca, nos damos cuenta de que hay demasiada variación utilizando el mismo n y T. Esto se debe a que la cota tiene mayor efectividad cuando el algoritmo ya encontró la mejor solución o una solución cercana a esto y la distribución de los datos no nos asegura que la encontremos rápidamente, el peor caso seria que se encontrara al final del recorrido DFS donde la cota no tendría ningún efecto.

Para medir este nuevo efecto, se añadió una nueva medida, calcular la incertidumbre de cada punto en el grafico que representa 10 tiempos con los mismos valores de n y T para tener clara la variación y también estudiar el peor caso que veremos al final.

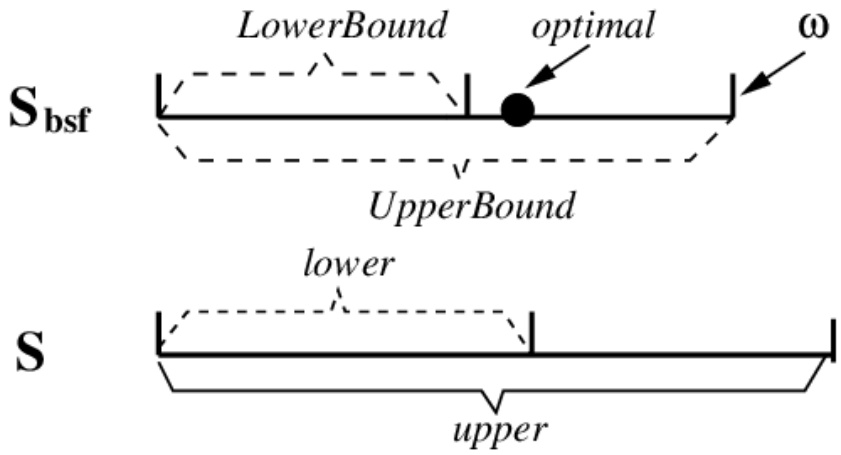
Al seguir buscando una mejora, una nueva idea sería intentar encontrar la solución viable en la primera bajada del árbol de decisión y acotar la búsqueda basada en eso. Para lograr esto se decidió tomar el criterio goloso como siguiente paso ya que este nos asegura una buena solución que por probabilidad puede estar cerca de la solución final dentro del árbol, aunque este requiriera la implementación de la estructura de datos Tema proveniente del algoritmo goloso.



Como se puede ver esto mejoro el tiempo de ejecución significativamente, pero se hace más claro el “caos” que se escondía anteriormente, la cota es ineficiente en reducir el área de búsqueda.

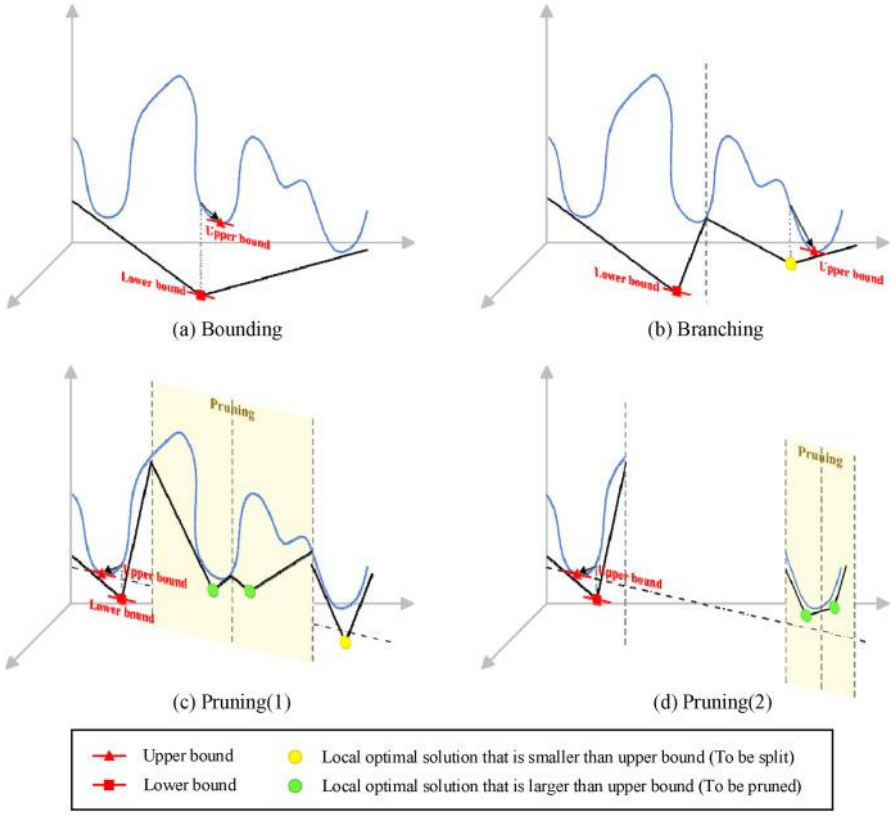
Aquí es cuando me tope con el concepto de B&B (Branch-and-Bound), un método para resolver problemas de combinatoria encontrando la solución óptima mediante Branching (Ramificación), Bounding (Acotamiento) y Pruning (Poda). La forma en la que estaba siendo resuelto el problema era usando en método B&B, una implementación ineficiente pero efectiva.

Adentrándonos en esta nueva información, encontramos 2 tipos de cotas que sirven para estos tiempos de problemas de optimización: cota superior (Upper bound) para problemas de maximización y cota inferior (Lower bound) para problemas de minimización. La cota inferior esta implementada en la forma del puntaje máximo encontrado y la cota que fue implementada anteriormente entra en la categoría de cota superior y esta será la que intentaremos mejorar.



Como se puede ver en esta representación visual nuestro tipo de problema es representado por S_{bsf} donde la cota inferior se representa comúnmente por el símbolo O este es el puntaje máximo encontrado que, al utilizar el criterio goloso, de primera será siempre la solución golosa y acota nuestro margen de búsqueda desde abajo, es decir, estamos asumiendo que solo se puede mejorar desde ahí y $remainingSum(i)$ es nuestra cota superior que es representada por ω.

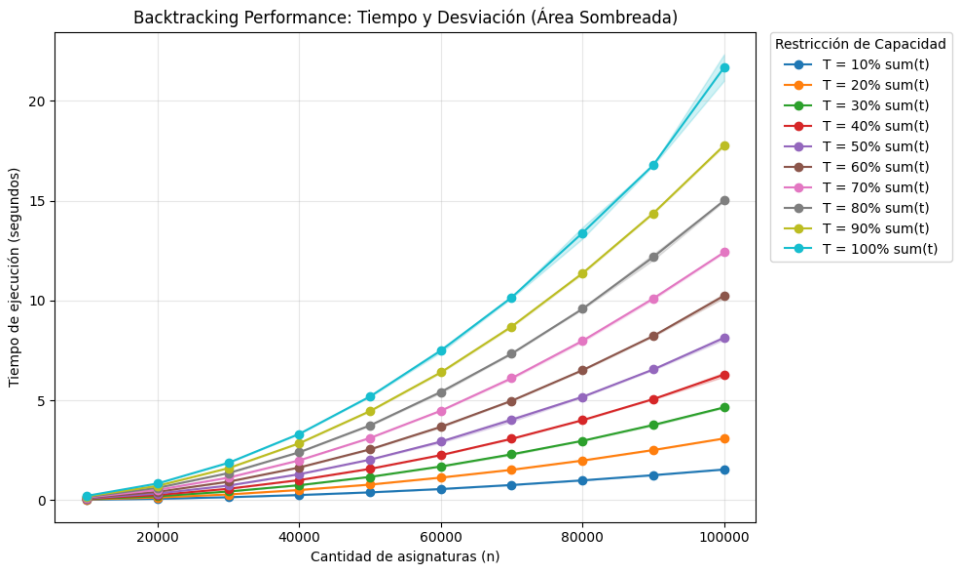
Gráficamente:



Ahora nos queda acercar lo máximo posible ω al optimo real sin saber cuál es.

Hay un problema que podemos utilizar como ω, el problema de la mochila fraccionaria que es tipo P, por ende, se puede solucionar en tiempo polinomial. El método goloso soluciona este problema óptimamente. Como ordenamos el arreglo de Temas nos saltamos ese paso y solo verificamos la solución y la comparamos con nuestro O.

Si $\omega \leq O$ entonces podamos esa rama ya que esto dicta que toda solución dentro de esta rama está fuera de la zona de acotamiento.



Aquí ya fuimos necesitando un mayor espacio para el stack de recursión, normalmente en Windows es de $1\ MiB$ y en GCC nativo es de $8\ MiB$, ambos quedaron pequeños ante estos valores de n y hubo que aumentar el stack a $1\ GiB$.

Para aumentar el tamaño del stack se tuvo que compilar el código con:

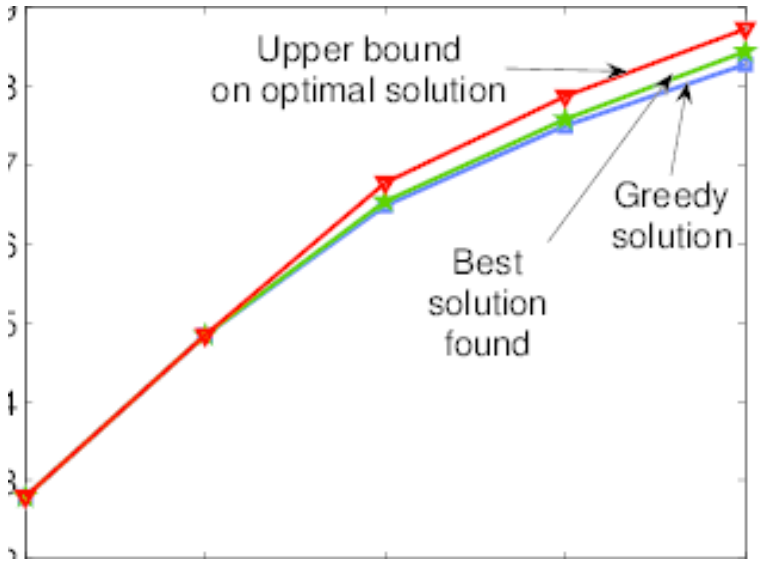
'gcc test_backtracking.c -o test_backtracking -O0 "-Wl,--stack,1073741824"'

Aumentamos el tamaño del stack a $1\ GiB$ para todas las pruebas en adelante.

Ahora el algoritmo es magnitudes más rápido, con una complejidad en el caso promedio $O(n^2)$ pero el peor caso $O(2^n \cdot n)$ empeoro debido a la implementación de la solución del problema de la mochila fraccionaria (la cota superior ω) tiene una complejidad $O(n)$ por cada nodo del árbol porque ya tenemos los temas ordenados por el criterio goloso. Podemos disminuir el tiempo que toma obtener ω a $O(\log n)$ por nodo con una Binary Search.

Antes de esto hemos de notar que lo que está pasando es que el algoritmo acota la búsqueda de esta manera:

$$Solucion\ Greedy \leq Zona\ de\ busqueda \leq Subproblema\ de\ la\ mochila\ fraccionaria$$



Esto podemos demostrarlo matemáticamente sabiendo que:

$$v_{greedy} \leq v_{opt} \leq v_{frac}$$

Donde, como ya sabemos:

$$\frac{1}{2} \leq \frac{v_{greddy}}{v_{opt}} \leq 1$$

Entonces estamos acotando por:

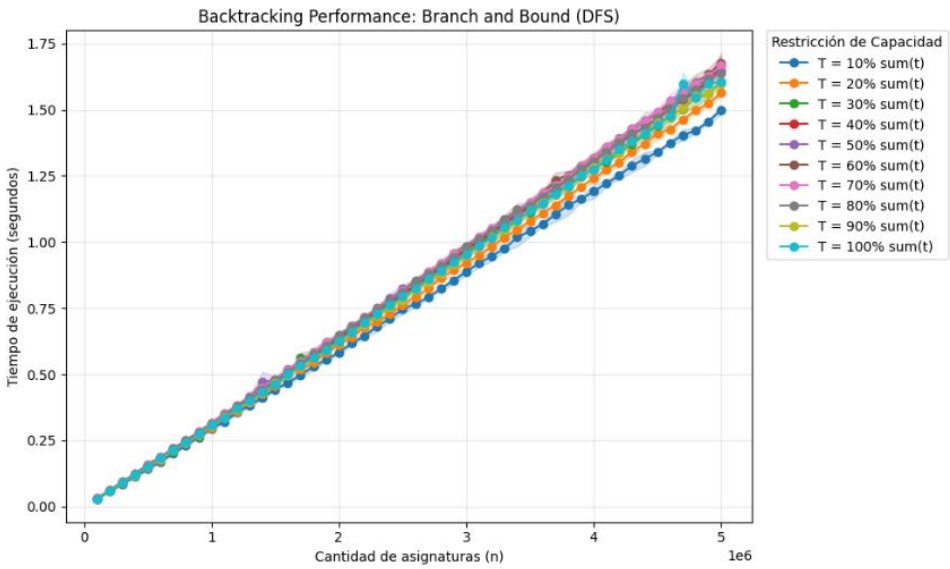
$$\frac{v_{opt}}{2} \leq Zona\ de\ busqueda \leq v_{frac}$$

Para lograr esta igualdad tenemos que evaluar el caso del "Elefante de oro" visto antes justo antes de empezar el backtracking.

La forma en la que se implementara todo esto es la siguiente:

1. Se asigna memoria para guardar la solución final (bool* sol) y para ir creando las permutaciones (bool* perm), se utiliza tipos de datos booleanos para que quepan mas en cache al llamarlos y evitar la frecuencia de ***cache misses.***
2. Se crea el arreglo auxiliar de Tema's.
3. Se ordena este arreglo por ratio.

4. Se inicializan 2 arreglos de sumas de prefijos (Prefix Sums) de t y p ordenados para reducir el costo de obtener la solución golosa a una Binary Search
5. Se verifica la solución golosa como cota inferior asignando ese valor a *max_value (O).
6. Se inicial el recorrido DFS en el árbol de decisión, y por cada nodo recorrido se compara la solución de la mochila fraccionaria usando los temas que quedan y si la solución obtenida es menor a nuestra cota inferior (*max_value), esta rama se poda por estar fuera de la zona de búsqueda.
7. Se construye la solución final.



Esta implementación mejora de gran manera el caso promedio $O(n \log n)$. El análisis para llegar a esta conclusión es el siguiente:

Fase de preparación	*Complejidad*
Inicializar arreglos y asignar valores iniciales	$O(n)$
Ordenamiento qsort()	$O(n \log n)$
Cota inferior ($\boldsymbol{Binary\ Search}$)	$O(\log n)$

Búsqueda en árbol de decisiones	*Complejidad*
Primer intento (Solución golosa)	$O(n)$
Cota superior (Mochila fraccionaria)	$O(\log n)$ por nodo recorrido
Recorrer ramas residuales	**Depende ($\boldsymbol{O(1)\ a\ O(2^n)}$)**

Recorrer las ramas restantes del árbol depende completamente de que la cota no nos de un resultado engañoso que fuerce al algoritmo a seguir buscando, cuando pasa esto

volvemos al caso donde solo teníamos poda y es peor aún ya que se ejecuta una Binary Search por cada nodo.

Análisis del peor caso del algoritmo backtracking

El problema viene cuando el acotamiento falla y nos deja solo con la poda de tiempo, la cota superior (solución al problema de la mochila fraccionaria) se calcula relajando la restricción de que los objetos no se pueden dividir.

Definamos los valores que afectan el tiempo de ejecución:

$$Solucion_{real} \rightarrow Valor\ de\ la\ solucion\ real\ y\ entera$$

$$Solucion_{fraccional} \rightarrow Valor\ de\ la\ solucion\ fraccional\ (Cota\ superior)$$

$$Gap = Solucion_{fraccional} - Solucion_{real}$$

Gap es la diferencia entre la mejor solución hipotética (fraccionaria) y la real, esto quiere decir que también es el tamaño del intervalo donde existe la incertidumbre de algoritmo.

Lo que dicta nuestra cota es que un subárbol solo puede ser podado si su limite superior truncado es menor o igual al récord actual.

$$\lfloor Solucion_{fraccional} \rfloor \leq Solucion\ *_{real}$$

** refiere a la solución local del subárbol.*

El peor caso absoluto ocurre cuando logramos diseñar una instancia donde el Gap es lo suficientemente grande como para que la condición de poda sea **matemáticamente inalcanzable** en la inmensa mayoría de los nodos del árbol y al suceder esto, causa que la zona de búsqueda que diseñamos quepa mayoritariamente o incluso completamente dentro del Gap.

Ej.

Imaginemos un caso simétrico, estos caen dentro del problema planteado.

$$t_i = 2\ para\ todo\ i \in [1, n]$$

$$p_i = 2\ para\ todo\ i \in [1, n]$$

$$T = N + 1$$

Ahora sabemos que la solución óptima real tomara exactamente $\frac{n}{2}$ temas.

$$Solucion_{real} = 2 \times \frac{n}{2} = n$$

Calculando ω para el nodo raíz. Como todos los temas tienen ratio 1, llenaremos la capacidad T.

$$Solucion_{fraccional} = T \times 1 = N + 1$$

La condición de poda falla:

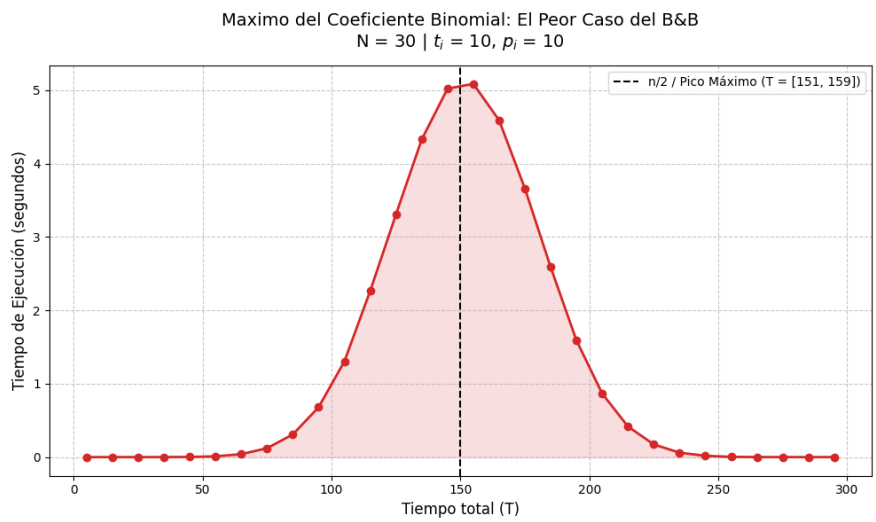
$$\lfloor N + 1 \rfloor \leq N \rightarrow N + 1 \leq N \; es \; Falso$$

El algoritmo no podara la rama derecha de la raíz ya que no tiene información suficiente para saber si no existe una solución con valor $N + 1$ en este subárbol.

Este problema se masifica al darnos cuenta de que esta condición se hereda a casi todos los nodos. Para saber cuántos nodos serán explorados por el algoritmo, usamos la formula el coeficiente binomial para responder la pregunta ¿De cuantas formas puedo elegir k temas de un total de n?

$$C(n,k) = \binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

Y el coeficiente binomial se maximiza cuando elegimos aproximadamente la mitad de los temas.



Una buena aproximación de esto es la aproximación de Stirling. Utilizaremos la aproximación del coeficiente binomial central que aumenta su precisión a medida que n aumenta.

$$\binom{n}{\frac{n}{2}} \sim \sqrt{\frac{2}{\pi n}}\, 2^n \approx Nodos \; hoja \; finales \; visitados$$

Esta es la cantidad de soluciones iguales a la que es capaz de llegar el algoritmo sin parar en este caso.

$$Nodos\ totales = Nodos\ hoja\ visitados + 2^{\frac{n}{2}-1}$$

$$Nodos\ totales = 2^n(\sqrt{\frac{2}{\pi n}} + \frac{1}{2})$$

Con esta información ahora si podemos asegurar que el peor caso es de $O(2^n \log n)$ porque bajo notación asintótica:

$$O\left(\frac{2^n}{\sqrt{n}} + 2^n\right) \times O(\log n) = O(2^n \log n)$$

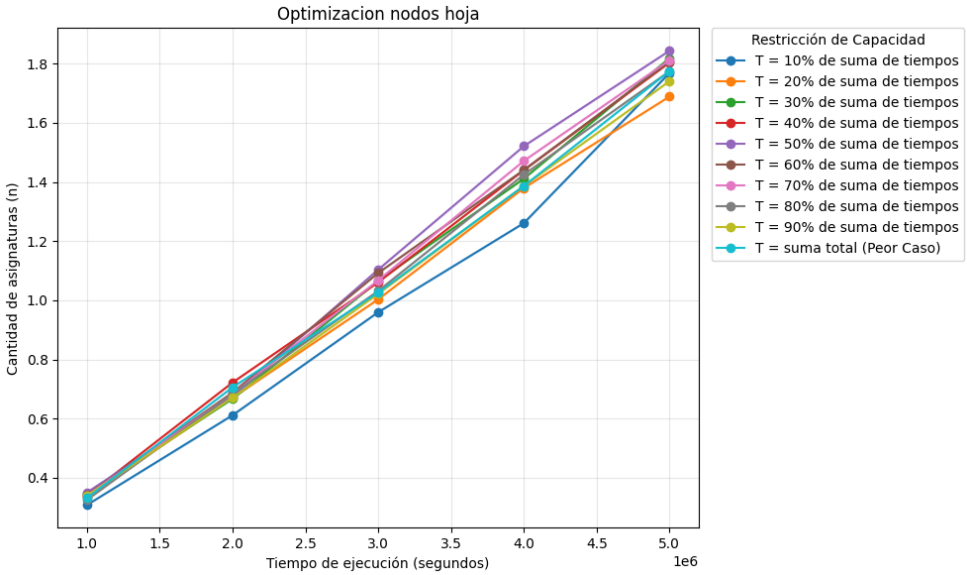
La fórmula creada para aproximar la cantidad de nodos totales también nos dice que mientras mayor sea n, menos nodos tendremos que recorrer respecto al tamaño total del árbol.

$$\lim_{n\to\infty} 2^n(\sqrt{\frac{2}{\pi n}} + \frac{1}{2}) \to \lim_{n\to\infty} 2^{n-1}$$

Así que incluso en el peor caso jamás recorreremos todo el árbol.

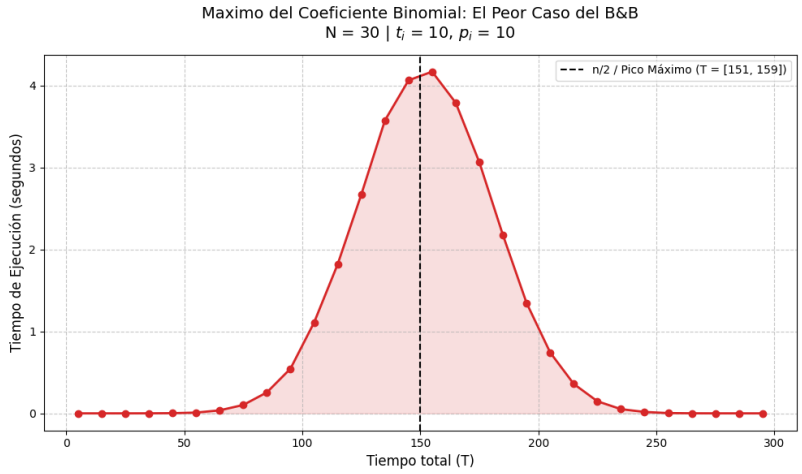
Podemos concluir que la técnica *Branch and Bound* colapsa cuando **la** relación entre peso y beneficio (el ratio) está diseñada para que la matemática continua (fracciones) difiera radicalmente de la matemática discreta (números enteros).

Este caso se puede optimizar mejorando la forma en la que el algoritmo gestiona los nodos hoja. Por ahora, se hacen las dos llamadas y se resuelven con el mismo caso base ($depth\ == n$), podemos mejorar esto haciendo que las hojas se resuelvan de una manera distinta ya que no hay mas temas que escoger, la mejor opción siempre es tomar el tema si cabe en el tiempo restante. Con esta mejora, disminuimos el numero de llamadas que se realizan en los peores casos.



Este es el caso promedio, aquí el tiempo de ejecución es casi igual ya que la cantidad de nodos hoja que recorre es menor.

En los peores casos donde se fuerza al algoritmo a buscar entre varias soluciones iguales



Aquí hay una mejora de aproximadamente un 18% ya que se recorren una mayor cantidad de nodos hoja como vimos antes. Mientras mayor sea la cantidad de soluciones que el algoritmo tenga que visitar, mayor será el impacto de esta optimización.

Es posible mejorar aún más la capacidad del algoritmo de prevenir y detener la búsqueda durante estos casos simétricos utilizando el método $Cutting\ Planes$ para crear Branch-And-Cut, pero esto entra en la rama de optimización

matemática Programación Entera (PE) y se encuentra muy fuera de mis conocimientos actuales.

Conclusión

Tras analizar e implementar los tres enfoques principales, la conclusión no es que exista un algoritmo superior a los demás, sino que cada método es una herramienta diseñada para un escenario de datos claro:

- Goloso:
  - Se hace un intercambio entre optimalidad y velocidad, resuelve un problema NP-Hard como un problema P.
- Programación Dinámica:
  - Se resuelve el problema desarmándolo en sus subproblemas óptimos y memorizándolos.
- Backtracking:
  - Al añadir la poda ya no trata de pasar por todo el árbol de decisiones, navega por el árbol de decisión con un "radar" para encontrar el mejor camino.

Escenario de datos	Restricciones	Enfoque superior
n bajo y T bajo	Ninguna	Programación Dinámica (Rápido, optimo y estable)
n masivo y T masivo	RAM y Tiempo de ejecución	Backtracking B&B (Recorre el árbol de decisión en caso promedio $O(n \log n)$)
Casos simétricos	Combinatoria	Programación Dinámica (Inmune a la simetría si T lo permite)
Tiempo real	Tiempos de ejecución instantáneos	Goloso modificado (Asegura un 50% del optimo)

Bibliografía

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
2. Martello, S., & Toth, P. (1990). *Knapsack problems: Algorithms and computer implementations*. John Wiley & Sons.
3. Wolsey, L. A. (2020). *Integer programming* (2nd ed.). John Wiley & Sons.
4. Horowitz, E., & Sahni, S. (1974). Computing partitions with applications to the knapsack problem. *Journal of the ACM*, *21*(2), 277–292. https://doi.org/10.1145/321812.321823
5. Pisinger, D. (1999). Core problems in knapsack algorithms. *Operations Research*, *47*(3), 395–405. https://doi.org/10.1287/opre.47.3.395Pisinger, D. (2005). Where are the hard knapsack problems? *Computers & Operations Research*, *32*(9), 2271–2284. https://doi.org/10.1016/j.cor.2004.03.002
6. Flajolet, P., & Sedgewick, R. (2009). *Analytic combinatorics*. Cambridge University Press. https://doi.org/10.1017/CBO9780511801655
7. Knuth, D. E. (1997). *The art of computer programming, Volume 1: Fundamental algorithms* (3rd ed.). Addison-Wesley Professional.
8. Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and combinatorial optimization*. John Wiley & Sons.
9. Schrijver, A. (1998). *Theory of linear and integer programming*. John Wiley & Sons.
10. Vazirani, V. V. (2001). *Approximation algorithms*. Springer. https://doi.org/10.1007/978-3-662-04565-7
11. Sahni, S. (1975). Approximate algorithms for the 0/1 knapsack problem. *Journal of the ACM*, *22*(1), 115–124. https://doi.org/10.1145/321864.321873
12. Ibarra, O. H., & Kim, C. E. (1975). Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, *22*(4), 463–468. https://doi.org/10.1145/321906.321909
13. Google. (2026). *Gemini* (Versión del 28 de febrero) [Modelo de lenguaje grande]. https://gemini.google.com/